UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

BORO RENTAL

ISPW PROJECT

Software Engineering and Web Design

Simone Boriani
0343024

INDEX

1. Software Requirement Specification 1
2. Storyboards 8
3. Design 12
3.1. Class Diagram 12
3.1.1. VOPC 12
3.1.2. Design Patterns 13
3.2. Activity Diagram 15
3.3. Sequence Diagram 17
3.4. State Diagram 17
4. Testing 18
5. Code 19
6. Video 22

1. Software Requirement Specification

1.1. Introduction

1.1.1. Aim of Document

The purpose of this document is to present the fundamental and specific requirements of the

“Boro Rental” system, a project developed for the Software Engineering and Web Design course,

Academic Year 2025–2026.

The document provides a comprehensive overview of the system’s objectives, core functionalities, technical requirements, and the rationale behind the design choices, to guide the development process.

Prepared by: Simone Boriani.

1.1.2. Overview of the defined system

"The system is designed with a **highly modular interface architecture**, supporting two distinct interaction modes that operate independently yet share the same underlying business logic. This duality is achieved through the **Abstract Factory pattern**, which ensures that the application's presentation layer remains decoupled from the functional core.

- **JavaFX Graphical User Interface (GUI):** This mode provides a comprehensive, interactive experience for users. Built with JavaFX and FXML, the GUI is designed for visual clarity and ease of use. It handles complex data rendering through table views, interactive pop-ups, and real-time state updates. The GUI layer is strictly limited to capturing user input and displaying data, delegating all processing tasks to the controller layer.
- **Command-Line Interface (CLI):** Designed for power users or resource-constrained environments, the CLI offers a streamlined, text-based navigation experience. Despite the simplified interface, it mirrors the complete feature set of the GUI, providing the same operational capabilities. The CLI architecture follows the same design principles, ensuring that switching between interfaces does not affect the system's state or behavior.

By centralizing view-side component creation through dedicated factories (such as the GraphicalSingletonFactory), the platform ensures that the system can switch between these two modes transparently at runtime. This approach not only enhances the platform's accessibility across different environments but also simplifies the maintenance of the presentation logic, as each UI family operates in a self-contained, logic-free environment.

The system supports the efficient and modern management of a virtual car rental platform, providing a seamless and user-friendly experience for both customers and administrators. The platform includes the following core functionalities:

- **User Authentication & Role Management:** Secure registration and login processes with distinct access levels for Guests, Registered Users, and AdministratorsPalle
- **Advanced Catalog Exploration:** A dynamic and interactive vehicle catalog that allows users to browse and filter cars based on multiple criteria (e.g., brand, model, price, fuel type, transmission, and category).
- **Vehicle Rental & Mock Payment Processing:** An end-to-end booking process integrated with a simulated payment gateway (Mock API) to handle secure and realistic virtual transactions.
- **Personal Garage Dashboard:** A dedicated space where users can monitor their active rentals, report technical issues, or manage early vehicle returns.
- **Integrated Notification System:** A real-time messaging and alert system that delivers automated system updates (e.g., rental confirmation.) and allows direct ticketing support between users and administrators.

- **Fleet Management (Admin):** Dedicated administrative tools to easily add new vehicles to the catalog and manage user support requests."

  There are three user types with different functionalities: **Guest**, **User** and **Administrator.**

1.1.3. HW e SW requirements

Software Requirements

Operating System	Windows 10 (64-bit), macOS 12+ o distribuzioni Linux (es. Ubuntu 20.04+).
DevelopmentKit (JKD)	Java 17 or higher.
Build Automation	Apache Mavaen 3.8+
User Interface	JavaFX SDK 25
Database	PostgreSQL v 42.7.8

Hardware Requirements

CPU	Dual-Core 2.0 GHz
RAM	2 GB
STORAGE	1 GB
Network	Required network connection for API

1.1.4. Related systems, Pros and Cons

RentalCars

Pros

- **Maximum savings**: Allows you to find the absolute lowest rate on the market thanks to price filters.
- **Global coverage**: You can find rental cars in practically any airport, station, or city in the world.
- **Advanced filters**: You can filter out rentals with disadvantageous fuel policies or overly high security deposits.

Cons

- **Traditional counter experience**: Upon arrival, you will still have to queue at the chosen company, handle paper forms, and face the risk of being "pressured" into buying extra insurance.
- **The "or similar" surprise**: You do not book a specific model, but a category (e.g., "Economy"). You might receive a different car from the one seen in the photo.

Virtuo

Pros

- **Zero queues and bureaucracy**: You manage everything from the app (document uploads, photographic check-in of the car's condition) and head straight to the parking lot.
- **Real and guaranteed cars**: You book the exact model you chose, not the classic "or similar". The cars are often mid-to-high end and very recent.
- **24/7 customer service**: Fast chat support directly integrated into the application.

Cons

- **Limited geographical coverage**: It is present mainly in large cities and major European airports. It does not comprehensively cover smaller tourist destinations.
- **Generally higher prices**: The premium experience and convenience come at a cost, making it difficult to find "low-cost" rates.

1.2. User Stories

US.1 - Car details visualization As a User, I want to view the details of a car, so that I can decide whether to rent it.

US.2 - Guest catalog browsing As a Guest, I want to view the catalog, so that I can evaluate the platform's offerings before deciding to register.

US.3 - Real-time cost calculation As a User, I want to see the applied pricing plan and the total cost in real time as I enter the rental days, so that I can evaluate the expense before confirming.

US.4 - Payment method selection As a User, I want to choose between paying with my account balance or via Stripe, so that I can use my preferred payment method.

US.5 - Global rental overview As an Administrator, I want to view all completed rentals, so that I can have a general overview of the platform's activity.

US.6 - Vehicle data modification As an Administrator, I want to be able to modify the details of a car already present in the catalog, so that I can keep the information up to date.

1.3. Functional Requiremtents

FR.1- The system shall provide the User with the detailed technical specifications and information of a single car selected from the catalog

FR.2- The system shall provide the Guest actor with read-only access to the entire catalog of cars available on the platform.

FR.3- The system shall calculate and provide in real-time the applicable contractual plan and the total estimated cost based on the number of rental days entered by the User.

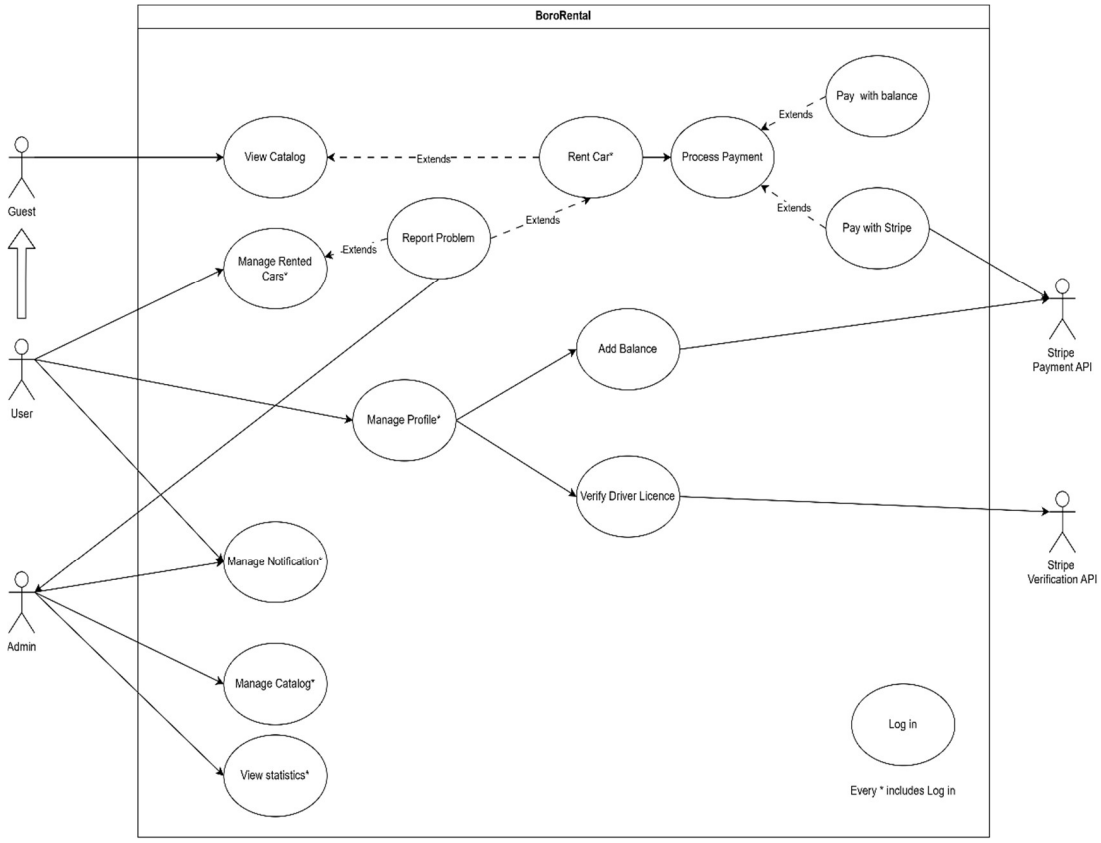
FR.4- The system shall provide the User with the ability to select the payment method for the transaction, choosing between the deduction from the internal account balance and the payment via the external Stripe service.

FR.5- The system shall provide the Administrator with access to a log containing the complete history of all rentals made on the platform.

FR.6- The system shall provide the Administrator with the ability to modify and update the data and characteristics of the cars already saved in the catalog database.

1.4. Use Cases

1.4.1. Overview Diagram



1.4.2. Internal Steps

UC-RENT A CAR

Primary actor - User

Main flow:

1. The User requests the system to rent a specific car identified from the catalog.
2. The system verifies the current availability of the requested car and display information.
3. The user provides the system with the expected nuber of rental days.
4. The system determines and applies the appropriate contractual plan based on the provided nuber of days, calculates the estimated total cost and provides it to the user.
5. The system verifies the validity of the driver's license.
6. The User selects the payment method (Interl balance or Stripe Service)
7. The system processes and authorizes the payment transaction baseod on chosen method
8. The system records the new rental , update the car's status and send a confirmation notification to the User.

Extensions:

2a. The requested car is no longer available or not found: The system displays an error message regarding the vehicle's unavailability and terminates the use case

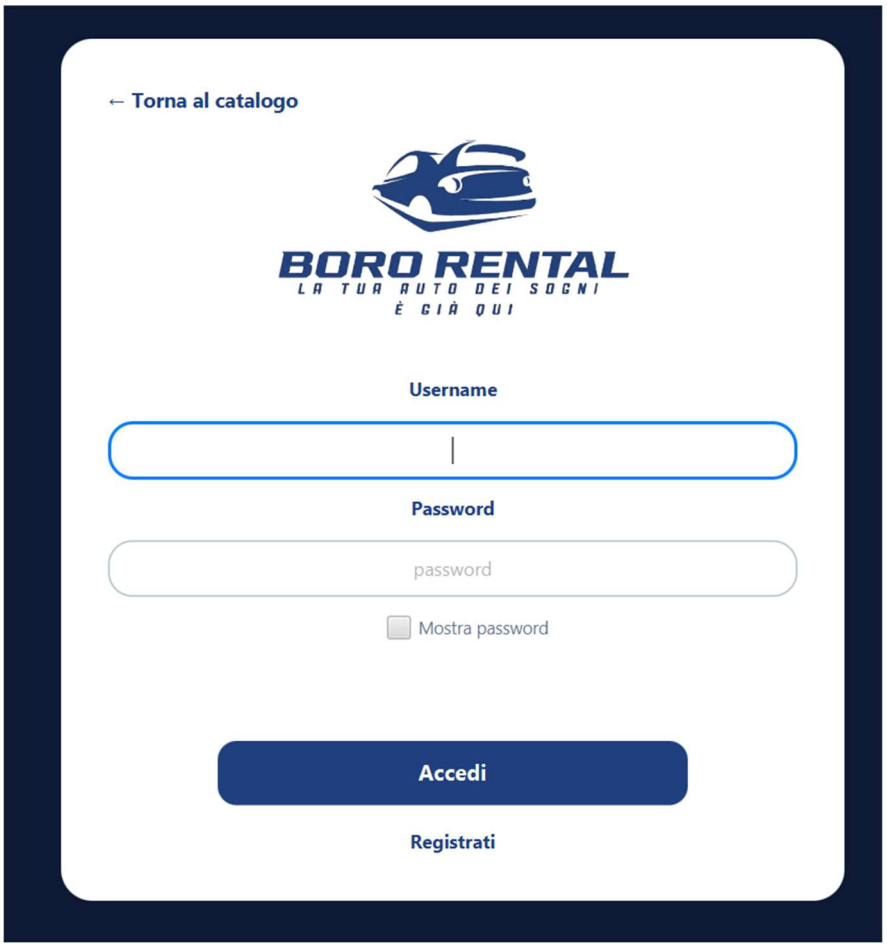
5a. The user provides an invalid or expired driver's license: The system displays an error message regarding the lack of valid driving requirements and terminates the use case.

7a. The payment transaction fails: The system displays an error message detailing the payment failure and asks to select an alternative payment method.

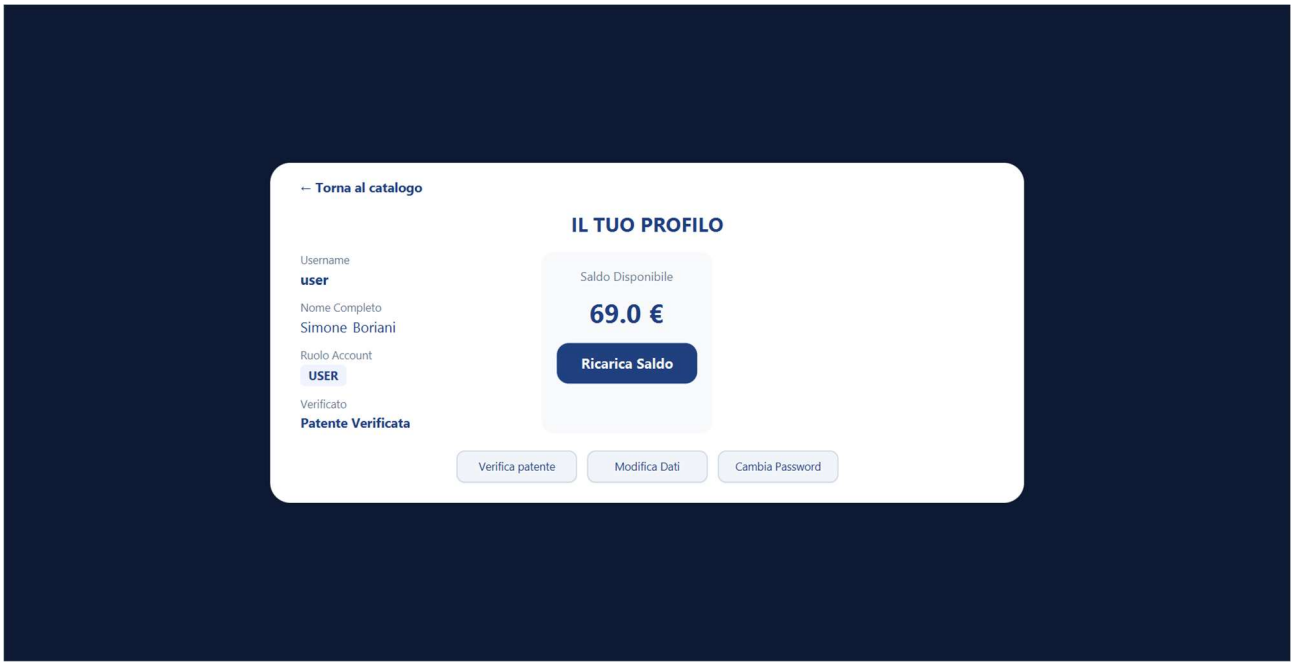
2. Storyboards

Catalog view:

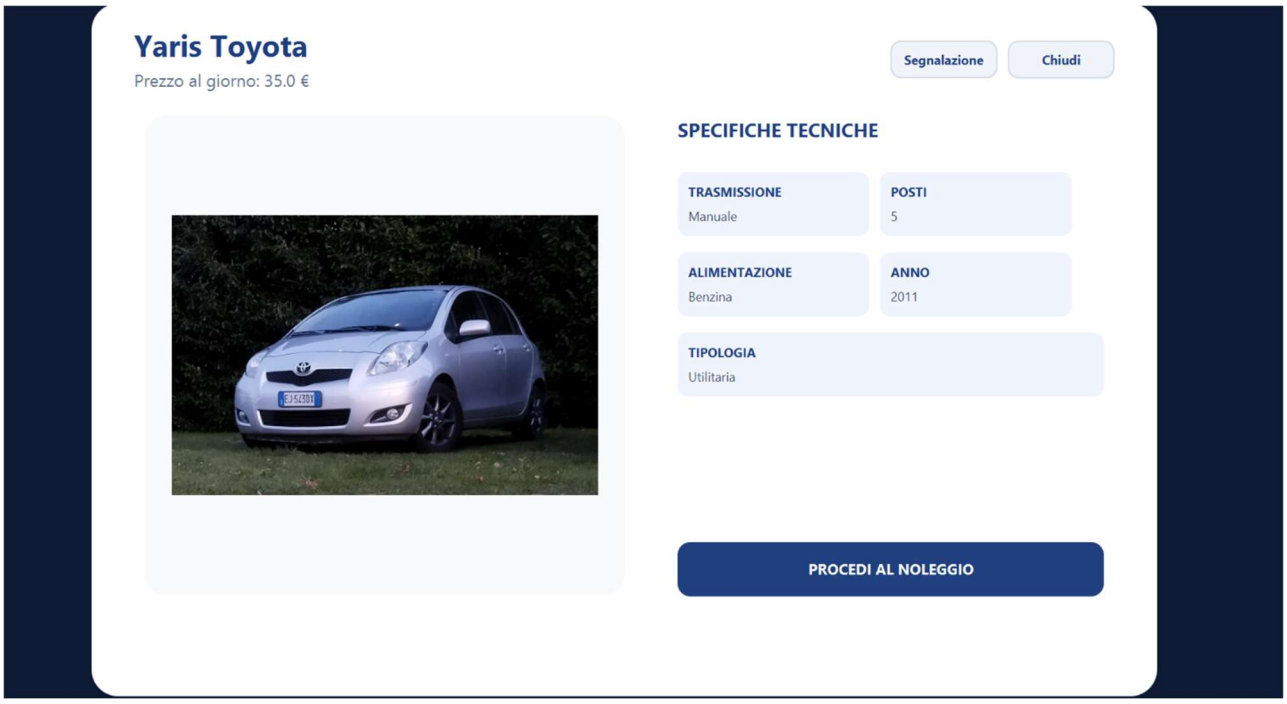
Log in:



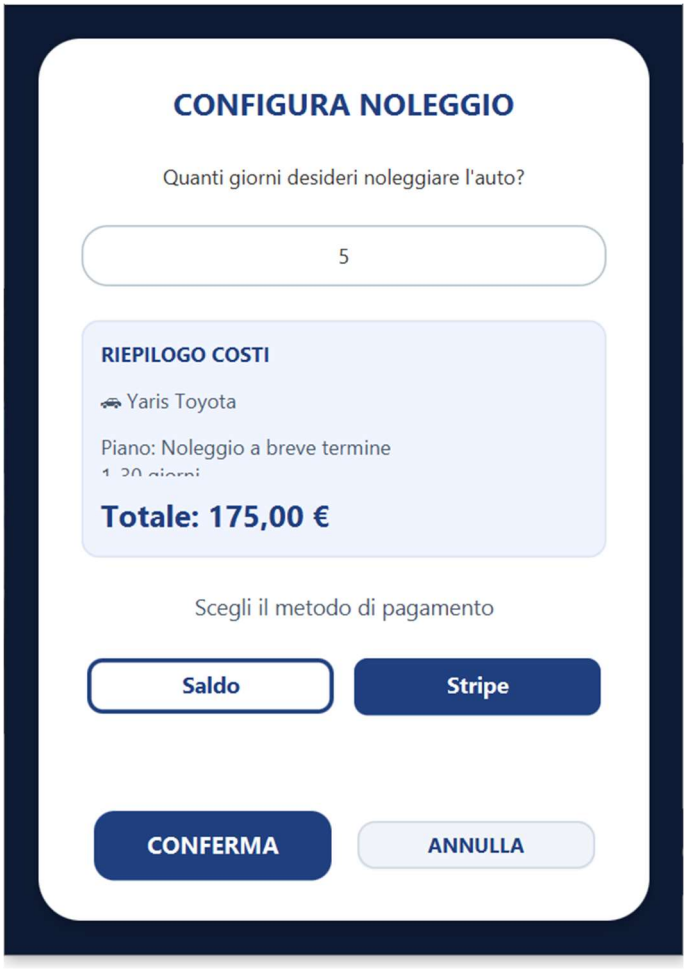
Profile Page:



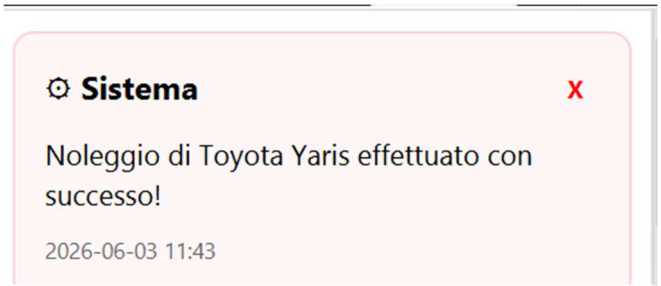
Car Details:



Rent configuration:



Notification:



Manage Rent:


← Torna al catalogo
IL TUO GARAGE
Gestisci i tuoi noleggi attivi e conclusi
Yaris Toyota
2011 • Benzina
2026-06-03 - 2026-06-08
TOTALE PAGATO: 175.0 €
Italia
Gestione Noleggio
Yaris Toyota
Prezzo giornaliero : 35.0 €
Stato: Noleggio Attivo
Termina Noleggio
Segnalazione
Seleziona un'auto per vedere i dettagli o chiudere il contratto

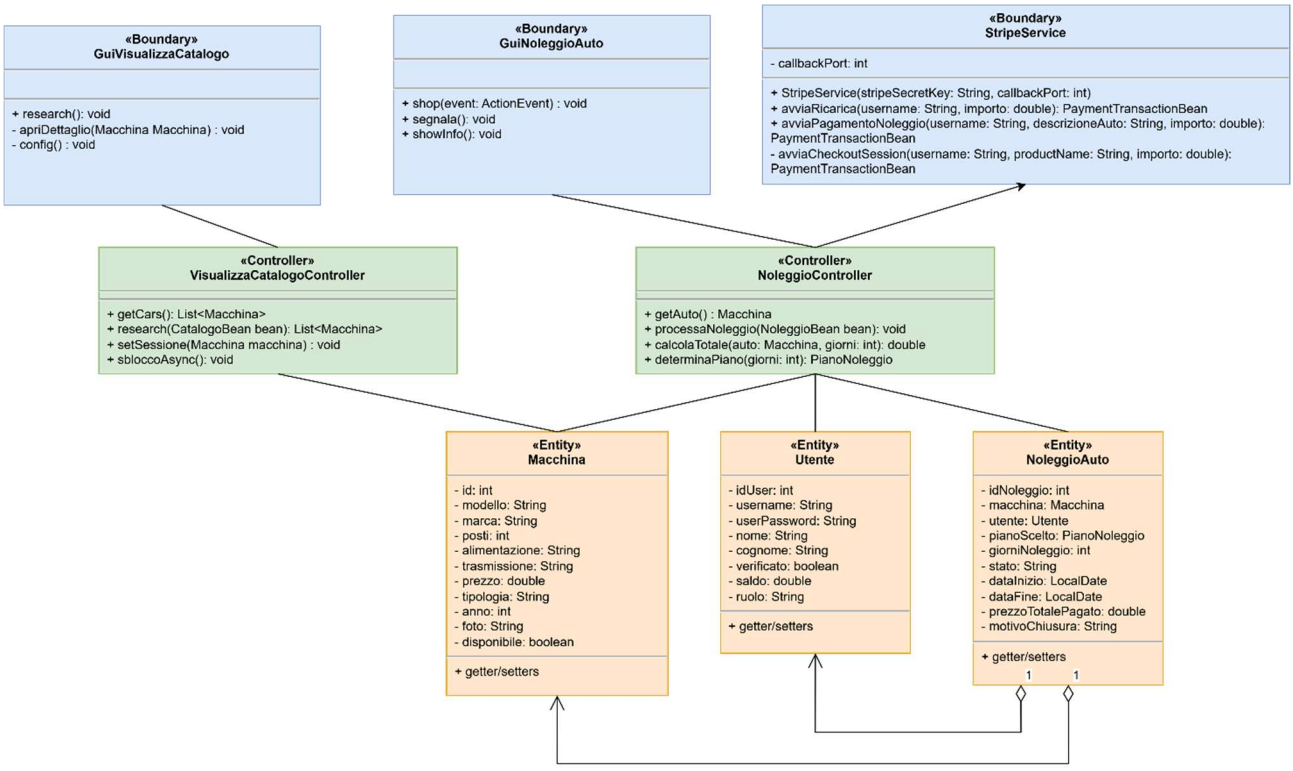
3. Design

3.1. Class Diagram

3.1.1. VOPC

The architectural design of Boro Renlat adheres to standard software engineering patterns to ensure scalability, maintainability, and a clear separation of concerns.

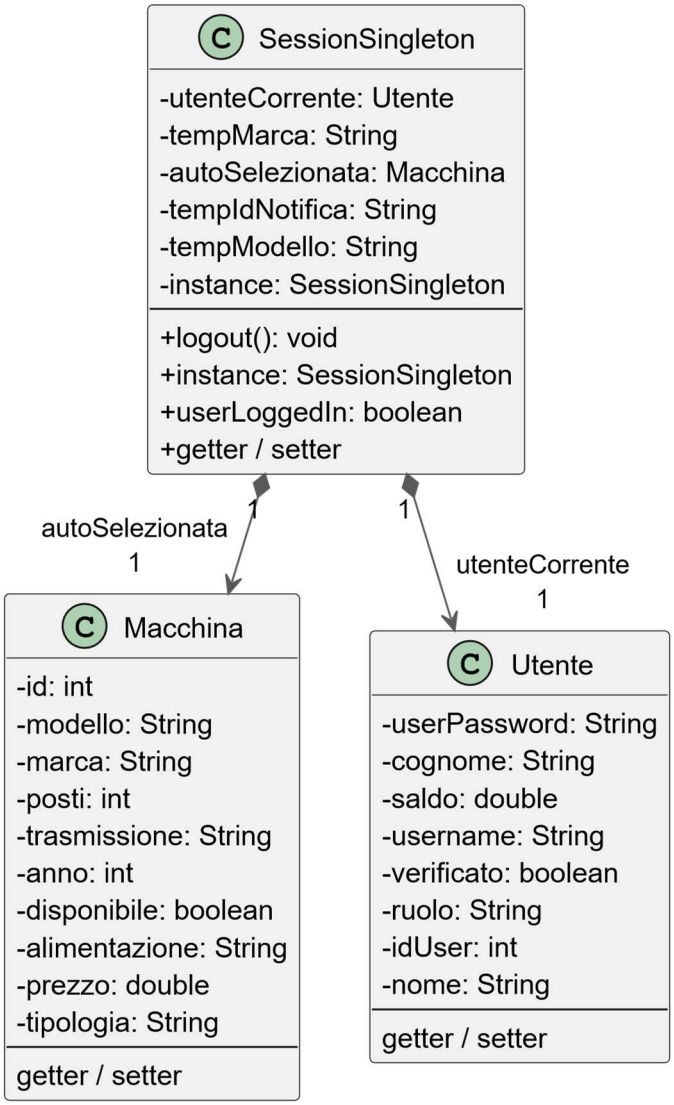
The following VOPC is rappresented by BCE pattern.



3.1.2. Design Patterns

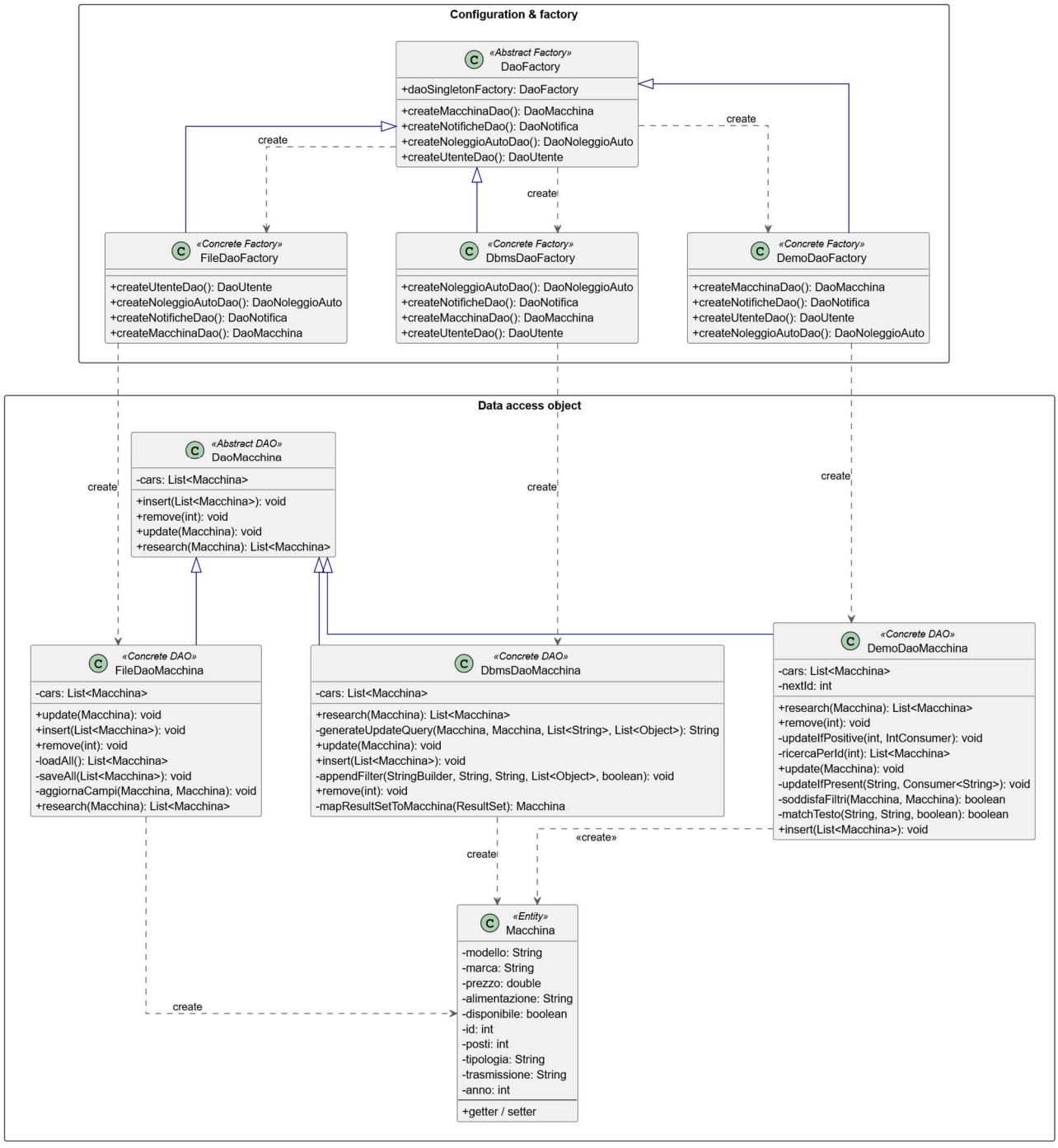
Singleton Pattern

In Boro Renal, we adopted the Singleton pattern to ensure that critical system components have a single, globally accessible instance. The SessionSingleton implements the pattern to rapresent the currently authenticated user , and the car seleceted from the catalog.



Abstract Factory Pattern

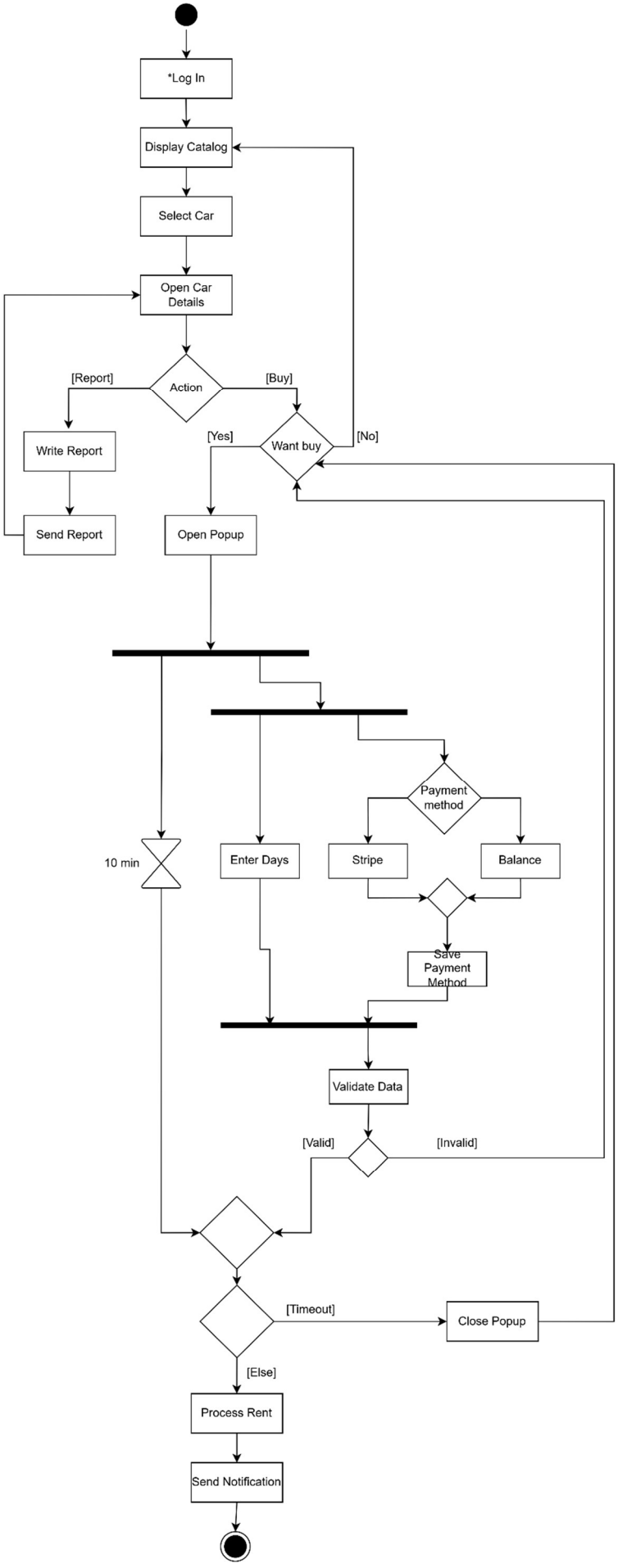
We applied the Abstract Factory pattern to manage the creation of the data access layer. The DaoFactory acts as an abstract interface for creating families of related Data Access Objects (DAOs).



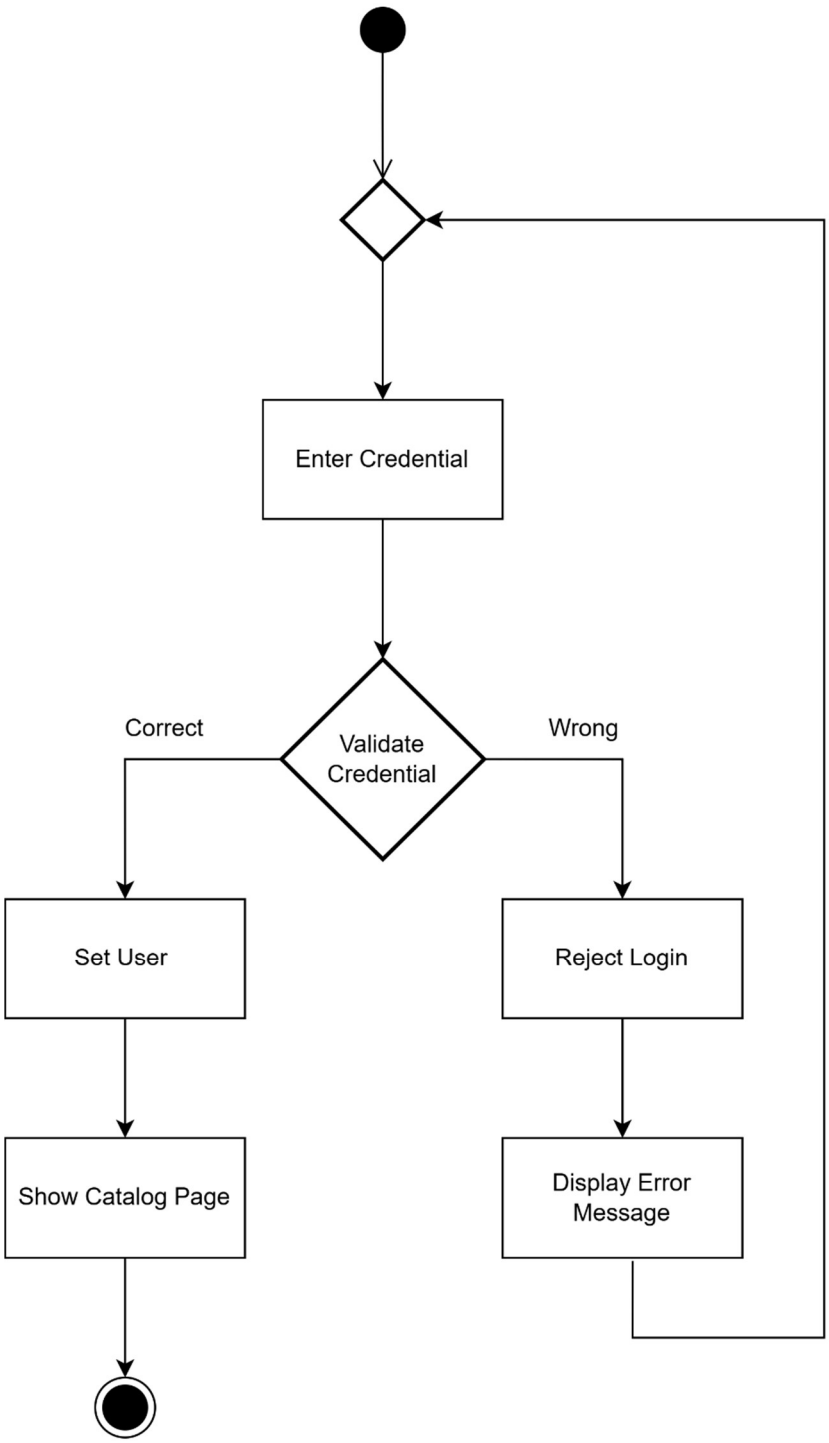
3.2. Activity Diagram

UC-Rent Car

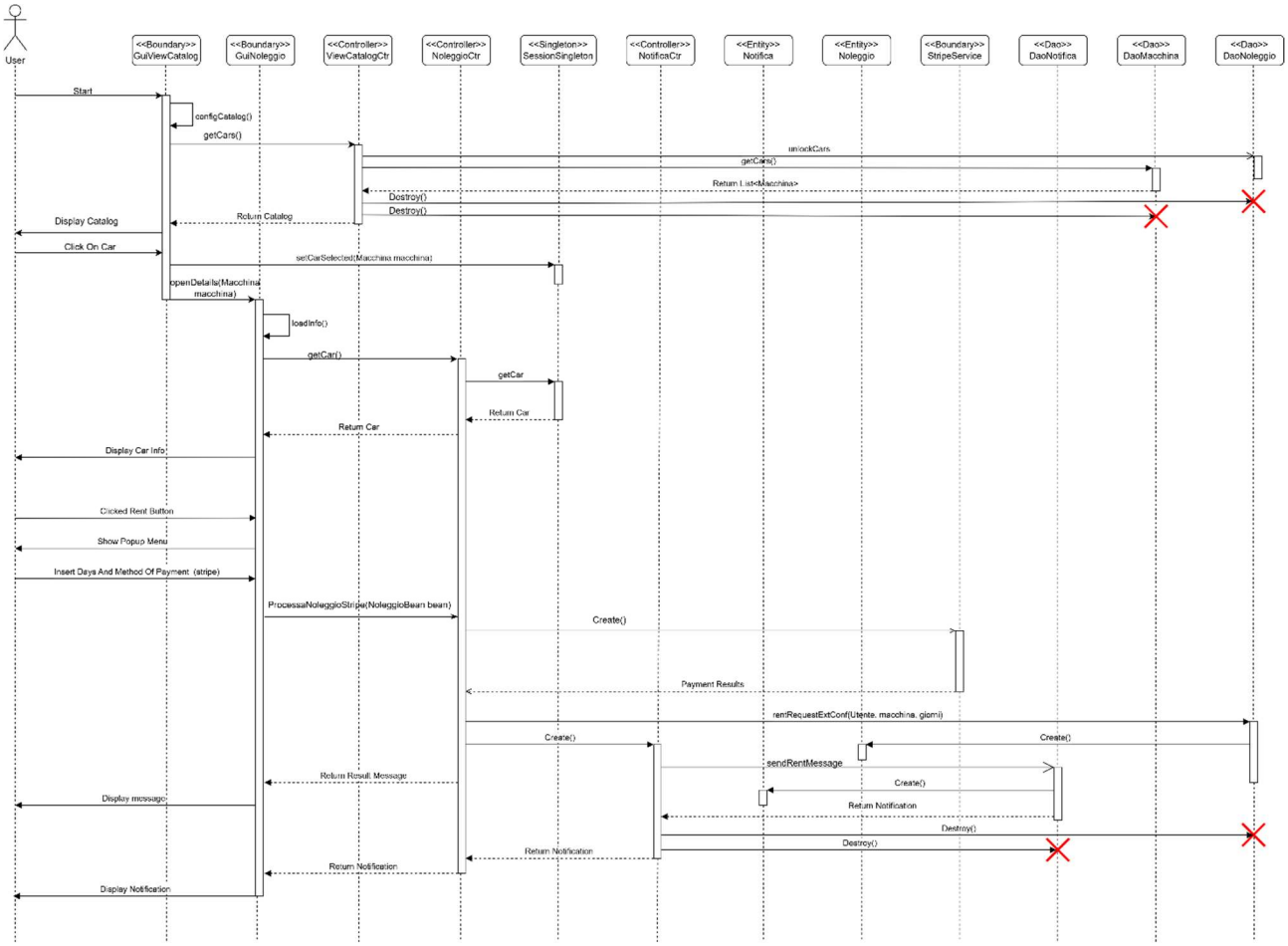
This Activity Diagram illustrates the execution flow of the **'Rent Car'** Use Case. It encompasses the core rental procedure as well as the essential preliminary prerequisites , the user authentication (login) .



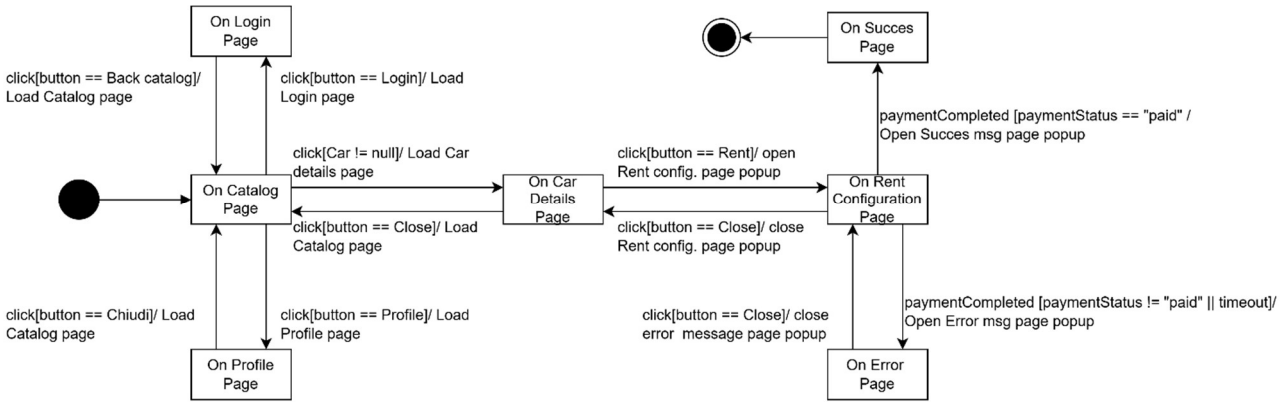
UC-LogIn


Enter Credential
Validate Credential
Correct
Wrong
Set User
Reject Login
Show Catalog Page
Display Error Message

3.3. Sequence Diagram



3.4. State Diagram



4. Testing

Login:

User registration role assignment We tested the logic responsible for assigning roles during the user registration process. The tests verify that the system correctly identifies the first registered user and grants them administrator privileges ("ADMIN"). Furthermore, the tests confirm that any subsequent user registrations are properly handled and assigned standard user roles ("USER"), ensuring appropriate access control right from system initialization.

User registration data validation We tested the validation mechanism for user registration inputs. The tests verify that mandatory credentials cannot be omitted, correctly rejecting registration attempts with null usernames or passwords by raising specific credential exceptions. Additionally, the tests ensure that the system prevents duplicate accounts by accurately identifying already existing usernames and aborting the registration process with appropriate error messages, thus maintaining data integrity.

User authentication verification We tested the authentication subsystem to ensure secure and accurate user login. The tests verify that the system robustly handles invalid login attempts by raising the appropriate exceptions when provided with non-existent usernames. Furthermore, the tests confirm that supplying an incorrect password for an existing, valid account is correctly detected and rejected with consistent error messages, preventing unauthorized access and maintaining system security.

Rent:

Car rental financial validation We tested the financial validation logic required for processing a car rental. The tests verify that users possessing a verified account and a sufficient balance can successfully complete a rental transaction without errors. Furthermore, the tests confirm that the system correctly intercepts and rejects transactions where the user lacks adequate funds, raising the appropriate exceptions to prevent financial inconsistencies.

Driver documentation verification We tested the document validation subsystem to ensure that only authorized users are permitted to rent vehicles. The tests verify that the system strictly enforces the requirement for a verified driving license. Rental attempts from unverified accounts are accurately rejected with specific document invalidity exceptions, ensuring the system adheres to safety and legal constraints prior to vehicle release.

Rental input data integrity We tested the system's resilience against invalid, incomplete, or logically flawed rental data. The tests verify that critical entities, such as the user or the selected vehicle, cannot be omitted, properly raising null reference exceptions when missing. Additionally, the tests ensure that temporal constraints are rigidly respected by correctly rejecting negative rental durations with appropriate error messages, thus maintaining the logical integrity of the rental contracts.

Early rental termination handling We tested the functionality governing the early closure of active vehicle rentals. The tests verify that when an ongoing rental is terminated ahead of schedule, the system accurately updates the contract's status to

reflect its completion. Furthermore, the tests ensure that the specific reason for the early closure is correctly logged and persisted in the backend data repository, ensuring precise tracking of vehicle lifecycles and contract histories.

Normal termination handling We tested the functionality governing the natural closure of active vehicle rentals. The tests verify that when a rental period concludes as originally scheduled, the system correctly identifies the contract as completed without requiring manual intervention. Furthermore, the tests ensure that the system logs the "Chiusura Naturale" reason and updates the vehicle's availability status in the backend data repository, confirming that the vehicle is correctly released and made available for future rentals.

Manage Catalog:

Vehicle catalog insertion verification We tested the functionality allowing the addition of new vehicles to the system's catalog. The tests verify that providing a completely and correctly populated vehicle entity successfully processes the insertion without generating exceptions. Furthermore, the tests confirm that upon saving, the newly added vehicle is accurately persisted and directly retrievable within the updated catalog list, ensuring consistent data availability.

Vehicle mandatory attribute validation We tested the validation logic concerning mandatory textual inputs during vehicle creation. The tests verify that critical identifiers, such as the vehicle's brand, cannot be omitted or left blank. Attempts to register incomplete vehicle profiles are accurately intercepted and rejected with specific argument exceptions, preventing the storage of unidentifiable entries and maintaining catalog integrity.

Vehicle numerical constraints validation We tested the data integrity checks applied to the numerical parameters of a vehicle. The tests verify that the system strictly enforces logical and business boundaries on attributes such as rental price, seating capacity, and manufacturing year. Specifically, the tests ensure that the system correctly rejects logically flawed inputs—such as negative rental prices, zero available seats, or historically improbable manufacturing years—by raising the appropriate exceptions with targeted error messages, thus ensuring only valid assets enter the system.

5. Code

The complete source code is available at:

https://github.com/SimoneBoriani/ISPW.git

Sonarcloud:

https://sonarcloud.io/project/overview?id=SimoneBoriani_ISPW

Exception:

The application adopts a structured paradigm for error management, integrating purpose-built runtime exceptions with defensive coding patterns. **These exceptions are propagated to the highest architectural layers, where they are caught and translated into explicit error messages displayed on the user interface.** This approach precisely isolates anomalous software states while safeguarding the resilience of the entire system.**Custom Exception Classes**

Application-specific exception classes, extending RuntimeException, are used to signal well-defined logical error conditions:

- **IncorrectCredentialExeption:** Raised when the user authentication or registration process fails. This encapsulates conditions such as providing incorrect passwords, submitting empty fields, or attempting to register an already existing username.
- **GenericSystemException:** Thrown to encapsulate infrastructure and system-level failures, such as database connectivity issues (SQLException), file I/O errors, or configuration loading failures. This prevents exposing underlying technical details while effectively bubbling up critical errors.
- **CarNotFoundException:** Raised when a catalog search or data retrieval yields no results based on the provided filters or ID, ensuring the UI can handle empty result sets gracefully.
- **DocsNotValidException:** Thrown when a user attempts to proceed with a rental request but their driver's license (patente) has not been successfully verified by the system.
- **PaymentFailedException:** Raised during the Stripe checkout process if the transaction setup is invalid, such as requesting a negative or zero amount for the account top-up.
- **ItemNotFoundException** and **NotificationErrorException:** Used to signal missing UI resources (like CSS files) and failures in saving system notifications, respectively.

Persistency:

In Boro Rental employs a flexible **Data Access Object (DAO)** pattern coupled with a **Factory** design. This architecture allows the system to seamlessly switch between different data storage mechanisms without altering the core business logic or the user interface. The system supports two primary persistence tiers

DEMO Level (In-Memory Persistence):

Designed for rapid prototyping, UI testing, and demonstrations, this volatile tier stores data exclusively in the application's RAM. It utilizes static **Java Collections** (e.g., ArrayList) to simulate database tables and basic integer counters to mock primary key auto-increments. All data is cleared upon application termination.

FULL Level (Non-Volatile Storage): Designed for persistent, long-term data retention, this tier is implemented through two distinct approaches:

- **File System:** A lightweight solution that reads and writes data to .csv files using Java's built-in I/O streams (java.io and java.nio.file). Java objects are manually serialized into comma-separated strings. Updates typically involve loading the data into memory, applying changes, and overwriting the target file.

- **DBMS (JDBC):** An enterprise-grade solution utilizing a relational database. It relies on active connections to interface with the database, executing SQL commands via secure PreparedStatement objects to prevent SQL injection. Crucially, it leverages strict **Database Transactions** (using commit and rollback) to guarantee complete data integrity across complex, multi-step operations, such as processing a car rental and deducting user funds simultaneously.
  Addendum on DBMS Infrastructure: The JDBC persistence tier is powered by a **Dockerized PostgreSQL** database. Utilizing a Docker container for the database layer ensures that the persistence environment is completely isolated, highly portable, and easily reproducible across different machines, removing the overhead of manual local database server configuration.

Dao:

Architectural Requirement: Dual DAO Implementation (DBMS and File System)

This design requirement lies at the core of the Data Access Object (DAO) pattern and demonstrates the ability to build a decoupled and highly flexible software architecture.

The objective is to provide at least one system entity with two entirely different physical storage mechanisms, while exposing the exact same logical interface to the rest of the application. In this project, this requirement is successfully fulfilled across multiple entities, most notably the Macchina (Car) entity.

1. The Abstraction (Abstract Class): The foundation of this architecture is defining *what* data operations are permitted, abstracting away *how* they are executed. This is achieved through the abstract class DaoMacchina, which establishes the contract for standard CRUD operations (e.g., insert, research, getCars, update). The application's controllers, such as GestioneCatalogoController, interact solely with this abstraction and remain completely unaware of the underlying storage engine.

2. The DBMS Implementation: The first required implementation interfaces with a relational database. Managed by the DbmsDaoMacchina class, this version translates application commands into SQL queries. It utilizes a ConnectionHandler to establish the connection and strictly relies on PreparedStatement objects to map Java attributes to database records, ensuring robust protection against SQL Injection vulnerabilities.

3. The File System Implementation: The second implementation fulfills the same contract but persists data to local files. Handled by the FileDaoMacchina class, it manages state using a .csv file. Rather than executing SQL, it leverages standard Java I/O utilities (java.nio.file and PrintWriter) to load the CSV records into RAM, apply modifications to the Java objects, serialize them back into comma-separated strings, and overwrite the file to persist the updates.

Architectural Value and Justification The primary value of fulfilling this requirement is the practical demonstration of **Polymorphism** and the **Factory Method Pattern** (DaoFactory).

Implementing these dual versions ensures that the application's business and presentation tiers are completely storage-agnostic. By parsing the config.properties file, the DaoFactory dynamically instantiates either the DbmsDaoFactory or FileDaoFactory at runtime. Consequently, the system can seamlessly transition from a remote PostgreSQL database to a local CSV file setup by modifying a single configuration string, requiring zero code changes in the controllers or graphical user interfaces.

6. Video

https://www.youtube.com/watch?v=UCoJelxXcKM